RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ..NIRANJAN.S.....

ACADEMIC YEAR ..2026-27... SEMESTER .....V.....BRANCH..B.E.C.S.D.

UNIVERSITY REGISTER No. 211624104010

Certified that this is the bonafide record of work done by the above student in the IT23E Graphics and Multimedia Laboratory during the year 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on ..03/11/2026.....

Internal Examiner

External Examiner

Scanned with OKEN Scanner

Exp-1 BRESENHAM'S LINE DRAWING ALGORITHM

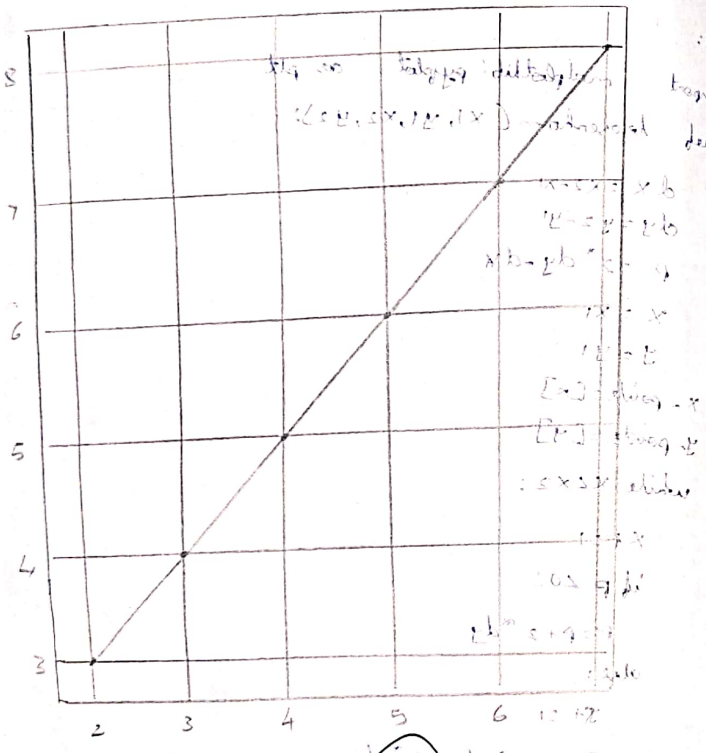
Aim:

To implement Brasenhams line drawing algorithm (|m|<1) in python for drawing a line when the slope.

Code:

```
import matplotlib.pyplot as plt
def brasenham(x1, y1, x2, y2):
    dx = x2-x1
    dy = y2-y1
    p = 2*dy-dx
    x = x1
    y = y1
    x_points = [x]
    y_points = [y]
    while x<x2:
        x+=1
        if p<0:
            p=p+2*dy
        else:
            y+=1
            p = p+2*dy-2*dx
        x_points.append(x)
        y_points.append(y)
    plt.plot(x_points, y_points, '60.')
    plt.title("Brasenham Line Drawing")
    plt.xlabel("x_axis")
    plt.ylabel("y_axis")
```

Scanned with OKEN Scanner




```
plt.grid(True)
plt.show()
```

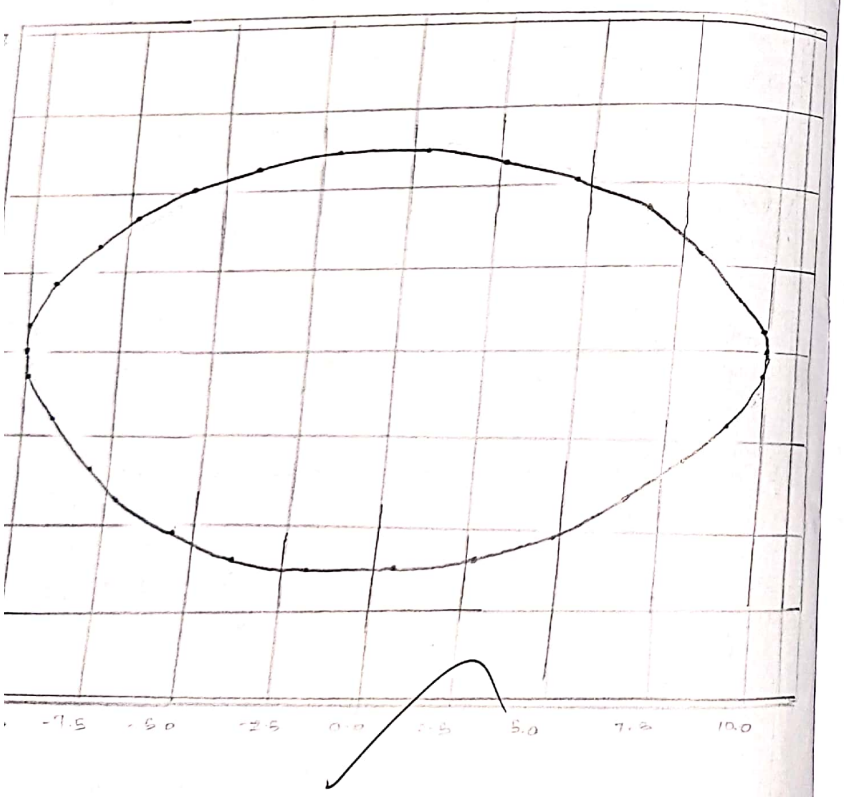
Example

```
x1, y1 = 2,3
x2, y2 = 10,7
bresenham(x1, y1, x2, y2)
```

14/8/26

Result:

The program was executed successfully. Bresenham line drawing algorithm for the input where slope |m| < 1.



Ex:-2 BRESENHAMS ELLIPSE DRAWING ALGORITHM

Aim:

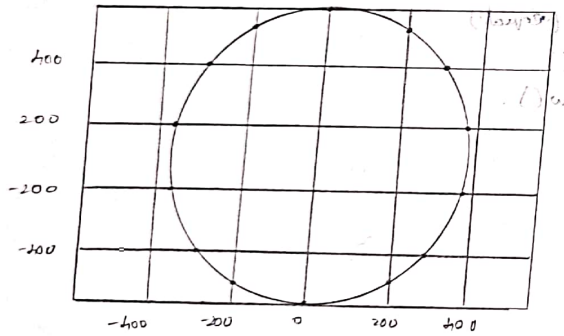
To implement Bresenhams ellipse drawing algorithm (ima in python for drawing a ellipse wh

Code:

```
import matplotlib.pyplot as plt
import numpy as np
t = np.linspace(0, 2*np.pi, 100)
x = 10 * np.cos(t)
y = 5 * np.sin(t)
plt.plot(x,y, 'bo-')
plt.axis('equal')
plt.grid()
plt.show().
```

Result: 14/8/26

The program was executed successfully. Bresenha ellipse drawing algorithm for the input where



EXP-3 BRESENHAM'S CIRCLE DRAWING Algorithm

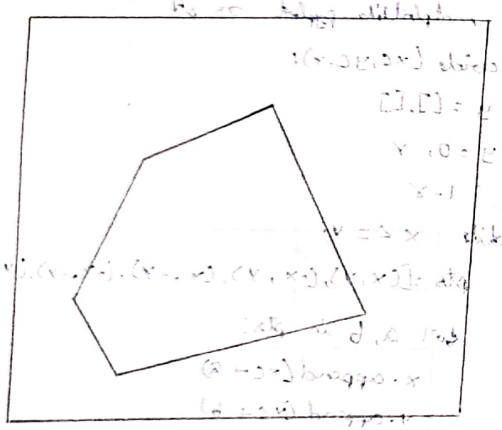
Aim:
To implement Bresenham's circle drawing algorithm in python for drawing circle.

Code:

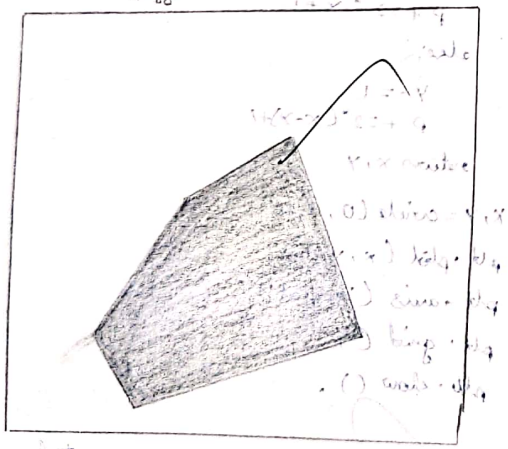
```
import matplotlib.pyplot as plt
def circle(xc,yc,r):
    x, y = [],[]
    x, y = 0, r
    p = 1-r
    while x <= y:
        pts =[(x,y),(-x,y),(x,-y),(-x,-y),(y,x),(-y,x),(y,-x),(-y,-
        for a,b in pts:
            x.append(xc+a)
            y.append(yc+b)
        x+=1
        if p<0:
            p+=2*x+1
        else:
            y-=1
            p+=2*(x-y)+1
        return x,y
x,y = circle(0,0,10)
plt.plot(x,y,'bo')
plt.axis('equal')
plt.grid()
plt.show().
```

Result:
The program was executed successfully. Bresenhams circle drawing algorithm for the input.

Polygon Outline (Bresenham)



Polygon Filled (Scan-line)



Polygon Using Bresenhams

Aim:

To implement the polygon using Bresenhams and color using scan line algorithm.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
def bresenham_line(x0, y0, x1, y1):
    points = []
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy
    while True:
        points.append((x0, y0))
        if x0 == x1 and y0 == y1:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x0 += sx
        if e2 < dx:
            err += dx
            y0 += sy
    return points
def draw_polygon(polygon_vertices):
    all_points = set()
    num_vertices = len(polygon_vertices)
    for i in range(num_vertices):
```

```
        x0, y0 = polygon_vertices[i]
        x1, y1 = polygon_vertices[(i+1) % num_vertices]
        line_points = bresenham_line(x0, y0, x1, y1)
        for p in line_points:
            all_points.add(p)
    return sorted(list(all_points))

def scan_line_fill(polygon_vertices, image_width, image_height):
    image = np.zeros((image_height, image_width), dtype=int)
    min_y = min(v[1] for v in polygon_vertices)
    max_y = max(v[1] for v in polygon_vertices)
    for y in range(min_y, max_y+1):
        intersections = []
        num_vertices = len(polygon_vertices)
        for i in range(num_vertices):
            p1_x, p1_y = polygon_vertices[i]
            p2_x, p2_y = polygon_vertices[(i+1) % num_vertices]
            if p1_y > p2_y:
                p1_x, p1_y = polygon_vertices[(i+1) % num_vertices]
                p2_x, p2_y = polygon_vertices[i]
            if p1_y > p2_y:
                p1_x, p2_x = p2_x, p1_x
                p1_y, p2_y = p2_y, p1_y
            if p1_y == p2_y:
                continue
            if p1_y <= y < p2_y:
                if (p2_y - p1_y) != 0:
                    x_intercept = p1_x + (y - p1_y) * (p2_x - p1_x) /
                                  (p2_y - p1_y)
                    intersections.append
                    (int(round(x_intercept)))
```

Scanned with OKEN Scanner

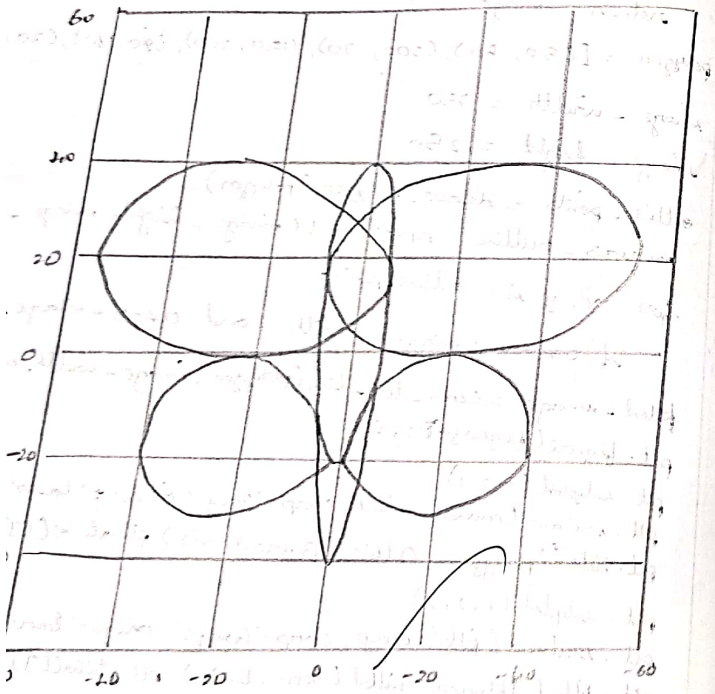
```
        intersections.sort()
        for i in range(0, len(intersections)-1, 2):
            x.start = intersections[i]
            x.end = intersections[i+1]
            for x in range(x.start, x.end+1):
                if 0<=x<image_width and 0<=y<image_height:
                    image[y,x] = 1
        return image
polygon = [(50, 50), (200, 70), (150, 200), (80, 180), (30,100)]
image_width = 250
image_height = 250
outline_points = draw_polygon(polygon)
canvas_outline = np.zeros((image_height, image_width))
for x, y in outline_points:
    if 0<=x<image_width and 0<=y<image_height:
filled_image = scan_line_fill(polygon, image_width, image_height)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(canvas_outline, cmap='Greys', origin='lower')
plt.title('Polygon Outline (Bresenham)') plt.xticks([]) plt.yticks([]
plt.subplot(1, 2, 2)
plt.imshow(filled_image, cmap='Greys', origin='lower')
plt.title('Polygon Filled (Scan-line)') plt.xticks([]) plt.yticks([])
plt.tight_layout()
plt.show()
```

Result:

Thus the polygon using midpoint alg is successfully executed.

Scanned with OKEN Scanner

Output:



Butterfly Using Midpoint Ellipse Alg.

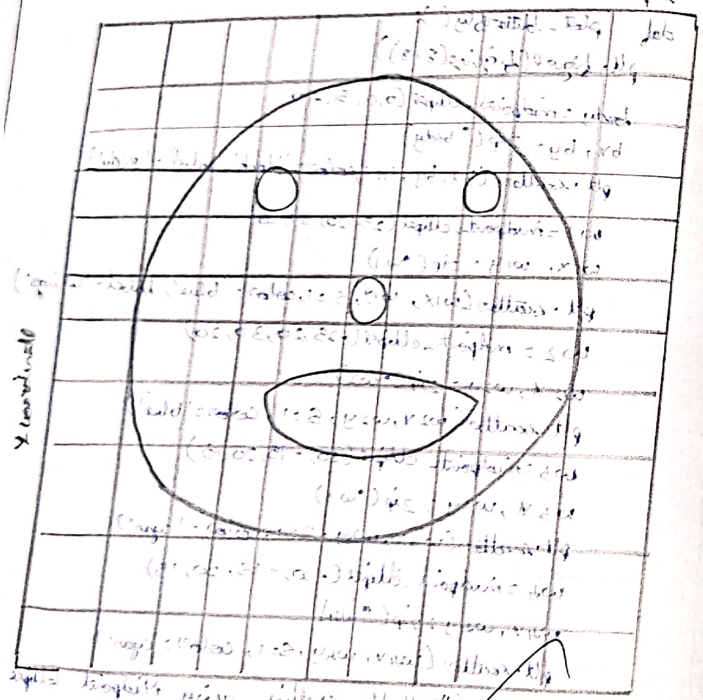
Aim: To draw the Butterfly using midpoint algorithm

Code:

```
import matplotlib.pyplot as plt
import numpy as np
def plot_butterfly():
    plt.figure(figsize=(8,8))
    body = midpoint_ellipse(0,0,5,40)
    bx, by = zip(*body)
    plt.scatter(bx, by, s=1, color='black', label='Body')
    w1 = midpoint_ellipse(25,20,30,20)
    w1x, w1y = zip(*w1)
    plt.scatter(w1x, w1y, s=1, color='blue', label='Wings')
    w2 = midpoint_ellipse(-25,20,30,20)
    w2x, w2y = zip(*w2)
    plt.scatter(w2x, w2y, s=1, color='blue')
    w3 = midpoint_ellipse(20,-15,20,15)
    w3x, w3y = zip(*w3)
    plt.scatter(w3x, w3y, s=1, color='cyan')
    w4 = midpoint_ellipse(-20,-15,20,15)
    w4x, w4y = zip(*w4)
    plt.scatter(w4x, w4y, s=1, color='cyan')
    plt.title("Butterfly Outline using Midpoint Ellipse Algorithm")
    plt.axis("equal")
    plt.show()
plot_butterfly()
```

Result:

Thus, the butterfly is successfully drawn.



Smiling face using midpoint ellipse Algorithm.

Aim:
To draw the smiling face using midpoint ellipse algorithm using python programming.

Code:

```
face_center_x = 100
face_center_y = 100
ellipses_face_data = [{'color': "yellow", 'cx': face_center_x, 'cy': face_center_y, 'rx': 40, 'ry': 40},
{'color': 'black', 'cx': face_center_x - 15, 'cy': face_center_y + 15, 'rx': 5, 'ry': 5},
{'color': 'black', 'cx': face_center_x + 15, 'cy': face_center_y + 15, 'rx': 5, 'ry': 5},
{'color': 'red', 'cx': face_center_x, 'cy': face_center_y - 15, 'rx': 20, 'ry': 8}
]
all_face_points = []
for ellipse_data in ellipses_face_data:
    ellipse_points = midpoint_ellipse(ellipse_data['cx'], ellipse_data['cy'],
ellipse_data['rx'], ellipse_data['ry'])
all_face_points.append({'color': ellipse_data['color'], 'points': ellipse_points})
print("Generated points for the smiling face.")

plt.figure(figsize=(8, 8))
for ellipse_data in all_face_points:
    color = ellipse_data['color']
    points = ellipse_data['points']
    x_coords = [p[0] for p in points]
    y_coords = [p[1] for p in points]
```

```
plt.scatter (x_coords, y_coords, color=color, s=10)
plt.title ('Smiling face using Midpoint Ellipse Algorithm')
plt.xlabel ('X-coordinate')
plt.ylabel ('Y-coordinate')
plt.gca().set_aspect ('equal', adjustable='box')
plt.grid (True)
plt.show()
```

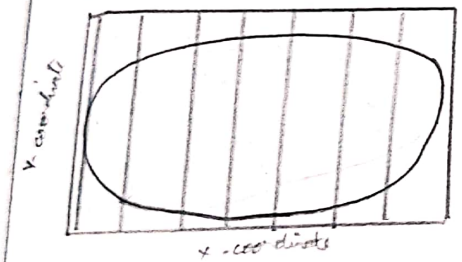
Result:

Thus the smiling face has been successfully drawn.

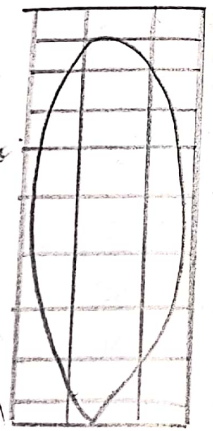
Scanned with OKEN Scanner

Output:

Horizontal Ellipse (rx = 30, ry = 20



Vertical ellipse (rx = 20, ry



Convert Horizontal & Vertical Ellipse into vertical

Aim: To draw the horizontal & vertical ellipse using Midpoint algorithm.

Boundaries

Code:

```
import matplotlib.pyplot as plt.
def midpoint_ellipse(xc, yc, rx, ry):
points=[]
x, y = 0, ry
p1 = round(ry**2 - rx**2 * ry + 0.25 * rx**2)
while (2 * ry**2 * x) < (2 * rx**2 * y):
    points.extend([(x + xc, y + yc), (-x + xc, y + yc), (x + xc, -y + yc),
(-x + xc, -y + yc)])
    x + = 1
    if p1 < 0:
        p1 = p1 + 2 * ry**2 * x + ry**2)
    else:
        y = y - 1
        p1 = p1 + 2 * ry**2 * x + ry**2 - 2 * rx**2 * y
p2 = round(ry**2 * (x + 0.5)**2 + rx**2 * (y - 1)**2 - rx**2 * ry**
horizontal_rx = 40
horizontal_ry = 20
horizontal_ellipse_points = midpoint_ellipse(cx, cy, horizontal_rx,
horizontal_ry) x_horiz, y_horiz = zip(*horizontal_ellipse_point
Vertical_rx = horizontal_ry
vertical_ry = horizontal_rx
vertical_ellipse_points = midpoint_ellipse(cx, cy, vertical_rx, v
ry) x_vert, y_vert = zip(*vertical_ellipse_points)
```

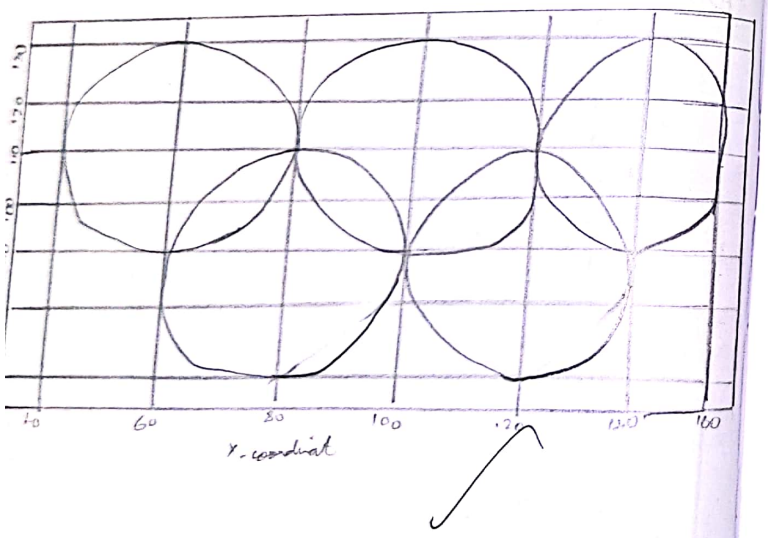
```
plt.figure(figsize(10,5))

plt.subplot(1,2,2)
plt.scatter(x_vert, y_vert, s=10, color='green')
plt.title(f'Vertical Ellipse (rx: {vertical_rx}, ry: {vertical_ry})')
plt.xlabel('x-coordinate')
plt.ylabel('y-coordinate')
plt.gca().set_aspect('equal')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Result: Thus the horizontal & vertical ellipse has been successfully executed.

Scanned with OKEN Scanner

Output:



Olympic Logo using

~~ring_radius=30~~

Aim: To ~~create~~ the olympic symbol using midpoint ~~intersections~~ alg.

Code:

```
ring_radius=20
logo_center_x=100
logo_center_y=100
top_row_y_offset = logo_center_y+ring_radius/2
bottom_row_y_offset = logo_center_y-ring_radius/2
rings_data=[
{'color':'black', 'cx': logo_center_x, 'cy':top_row_y_offset,'rx':ring_radius
{'color':'red', 'cx': logo_center_x+ 2*ring_radius,'cy': top_row_y
offset, 'ry': ring_radius 'ry': ring_radius},
'color': 'yellow', 'cx': logo_center_x - ring_radius, 'cy': bottom_
row_y_offset, 'rx' :ring_radius, 'ry' : ring_radius},
]
all_ring_points =[]
for ring in rings_data:
    ring_points = midpoint_ellipse(ring['cx'], ring['cy'],
    ring['rx'] ring['ry'])
print("Generated points for Olympic rings:").
```

```
plt.figure(figsize(10,6))
for ring_data in all_ring_points:
    color = ring_data['color']
    x_coords = [p[0] for p in points]
    y_coords = [p[1] for p in points]
    plt.scatter(x_coords, y_coords, color=color, s=1)
plt.title('Olympic Logo using Midpoint Ellipse Algorithm')
plt.xlabel('x-coordinate')
plt.ylabel('y-coordinate')
plt.gca().set_aspect('equal', adjustable='box')
plt.grid(True)
plt.legend(bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()
plt.show()
```



Result:

Thus the olympic symbol has been successfully executed.

9. Scan Conversion of Polygon using flood fill

Aim:

To implement scan conversion of polygon using flood fill algorithm.

Program:

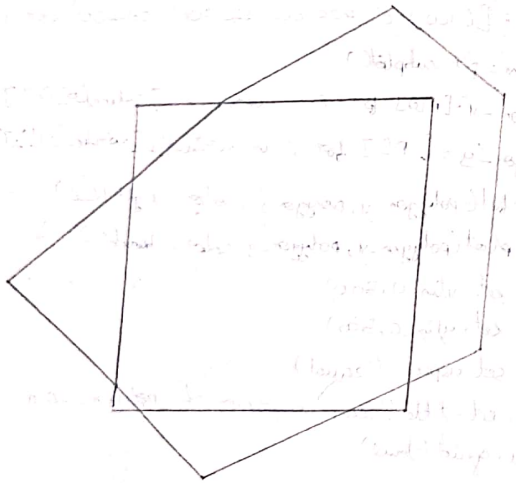
```
import numpy as np
import matplotlib.pyplot as plt.
height = 500 , width = 500
image = np.ones( height , width , 3), type = np.uint8) * 255
vertices = [( 100,100),(400,100),(350, 300),(250,200),(100,300)]
fig, ax = plt.subplots( )
polygon_x = [P[0] for P in vertices] + [vertices[0][0]]
polygon_y = [ P[1] for P in vertices] + [vertices[0][1]]
ax.fill( polygon_x, polygon_y , color = 'light blue')
ax.plot(polygon_x, polygon_y, color = 'black')
ax.set.xlim(0,500)
ax.set.ylim(0,500)
ax.set aspect ('equal')
ax.set.title ("scan conversion of polygon using
ax.grid (True)
```

Result:

Thus the implementation of scan conversion of polygon using flood fill algo is successfull.

Scanned with OKEN Scanner

Output:



10. Basic 2D transformations using 2D shapes

Aim:

To implement basic 2d transformations using 2D shapes using polyglot programming language.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
triangle = np.array([[2,2],[5,2],[3,5],[2,2]])
tx,ty = 2,1
translation = triangle + [tx,ty]
sx,sy = 2,2
scaling = triangle * (sx,sy)
angle = np.radians(45)
rotation_matrix = np.array([[np.cos(angle), np.sin(angle)],
                            [np.sin(angle), np.cos(angle)]])
rotation = triangle @ rotation_matrix
reflection = triangle * (1,1)

shx = 1
shear_matrix = np.array([[1, shx],[0,1]])
shear = triangle @ shear_matrix.T
spring = triangle @ shear_matrix.T
fig, axes = plt.subplots(2,3, figsize=(10,7))
axes[0,0].plot(triangle[:,0], triangle[:,1], 'bo-')
axes[0,0].set_title("original triangle")
axes[0,1].plot(translation[:,0], translation[:,1], 'ro')
axes[0,1].set_title("translation")
axes[0,2].plot(scaling[:,0], scaling[:,1],)
axes[1,0].plot(reflection[:,0], reflection[:,1],'r=')
```

Scanned with OKEN Scanner

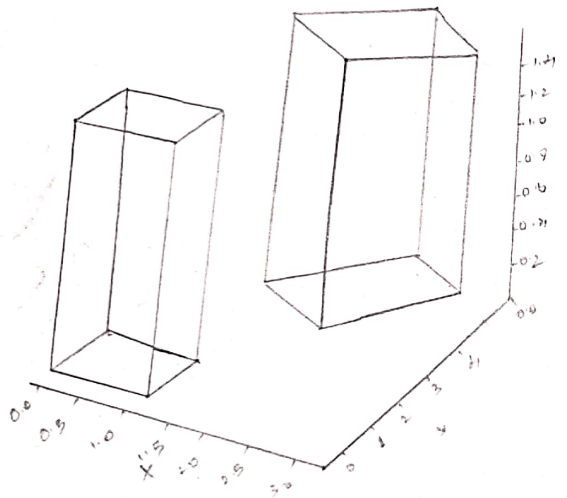
```
axes[1,1].set_title("Reflection")
axes[1,2].plot(shearing[1,0] shearing[:,1], shearing[:,0])
axes[1,2].set_title("shearing")
for ax in axes.flat:
    ax.axvline(0, color='black', linewidth=0.5)
    ax.axhline(0, color='black', linewidth=0.5)
    ax.grid(True)
    ax.set_aspect('equal')
plt.show()
```

Result:

Thus the implementation of 2D transformation using 2D shapes using python is successfull.

Scanned with OKEN Scanner

Output:



10) Polygon Clipping

Aim:

To write a program clips a polygon to a specified rectangular window using the sutherland hodgman polygon clipping alg and displays the clipped polygon.

Program:

```
import matplotlib.pyplot as plt.
LEFT, RIGHT, BOTTOM, TOP=0, 1,2,3.
def inside(p, edge, clip_win)
    x, y = p
    xmin, xmax, ymin, ymax = clip_win
    if edge == LEFT :
        return x >= xmin
    elif edge == RIGHT
        return x <= xmax
    elif edge == BOTTOM:
        return y >= ymin
    elif edge == TOP:
        return y <= ymax
def intersect (P1, P2, edge, clip_win)
    xmin, xmax, ymin, ymax, clip_win,
    x1, y1 = P1:
    x2, y2, P2
    if edge == LEFT
        x = xmin
    y = y1 + (y2-y1) * (xmin-x1)(x2 - x1)
    elif edge == RIGHT :
        x = xmax
        y = y1 + (y2-y1) * (xmax - x1) / (x2 - x1)
```

```
elif edge == BOTTOM:
    y = y_min
    x = x1 + (x2 - x1) * (y_min - y1) / (y2 - y1)
elif edge == TOP:
    y = y_max
    x = x1 + (x2 - x1) * (y_max - y1) / (y2 - y1)
return (x, y)

def clip_program(polygon, clip_win):
    output = polygon
    for edge in (LEFT, RIGHT, BOTTOM, TOP):
        input_list = output
        output = []
        if not input_list:
        if inside(P, edge, clip_win):
        if not inside(S, edge, clip_win):
            output.append(intersect(S, P, edge, clip_win))
        output.append(P)
    elif inside(S, edge, clip_win):
    return output
```

Result:

Thus the implementation of polygon clipping is successful and the output is verified.

12. 3D Transformations on Basic objects (Cube, Pyramid)

Aim:

To write a program that allows the user to perform 3D transformations on basic 3D objects like a cube or pyramid, and visualize the results.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Line3DCollection
def draw_edges(ax, vertices, edges, color='b'):
    lines = [[vertices[start], vertices[end]] for start, end in
edges]
def translation_matrix(tx, ty, tz):
    return np.array([[1, 0, 0, tx], [0, 1, 0, ty], [0, 0, 1, tz], [0, 0, 0,
def scaling_matrix(sx, sy, sz):
    return np.array([[sx, 0, 0, 0], [0, sy, 0, 0], [0, 0, sz, 0], [0, 0, 0, 1]])
def rotation_matrix_z(angle):
    rad = np.radians(angle)
    return np.array([[np.cos(rad), -np.sin(rad), 0, 0], [0, 0, 1, 0], [0,
def apply_transform(vertices, matrix):
    transformed = []
    for v in vertices:
        vec = np.array([*v, 1])
        result = matrix @ vec
        transformed.append(result[:3])
    return transformed
```

Scanned with OKEN Scanner

```
vertices = [(0,0,0), (1,0,0), (1,1,0), (0,1,0), (0,0,1), (1,0,1), (1,1,1), (0,1,1)]

edges = [(0,1), (1,2), (2,3), (3,0), (4,5), (5,6), (6,7), (7,4), (0,4), (1,5), (2,6), (3,7)]

T = translation_matrix (2,0),
S = scaling_matrix(1.5, 1.5, 1.5)
R = rotation_matrix_z(45)
transformed_vertices = apply_transform (vertices, T @ S @ R)

fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')
draw_edges (ax, vertices, edges, 'blue')
draw_edges (ax, transformed_vertices, edges, 'red')
ax.set_title ("3D Transformation of Cube")
ax.set_xlabel ('x')
ax.set_ylabel ('y')
ax.set_zlabel ('z')
plt.show()
```

Result:

The user was able to perform Translation, scaling rotation on a 3D cube. was successfully rendered and visualized.

Scanned with OKEN Scanner

13. Create and Render 3D objects using polygons and Apply Basic color & Shading Techniques.

Aim:

To create and render 3D objects using polygonal modeling and apply:

a) Flat shading

b) Basic color techniques.

Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,0,1],[0,1,0],[0,0,1],[1,0,1]])
faces = [[vertices[i] for i in [0,1,2,3]],
         [vertices[i] for i in [0,1,3,4]],
         [vertices[i] for i in [2,3,5,7]],
         [vertices[i] for i in [1,2,5,5]],
         [vertices[i] for i in [4,7,3,0]]]
```

Result:

3D polygonal objects were successfully created & rendered

Scanned with OKEN Scanner

4. Create a Multimedia Application that Integrates Images, Sound, Video in a simple User Interface

Aim.

To develop a simple multiplication application that integrates : Images, Audio, Video.

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import image, ImageTk
from playsound import playsound
import cv2
import threading
def play_audio():
    playsound('sample_audio.mp3')
def play_video():
    cap = cv2.VideoCapture('sample_video.mp4')
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Video", frame)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
def load_image():
    img = Image.open('sample_image.jpg')
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage(img)
```

```
panel.configure(image = img_tk)
panel.image = image = tk
window = tk.Tk()
window.title("Multimedia App")
btn_img = tk.Button(window, text="Show Image", command = load_Image)
btn_audio = tk.Button(window, text="Play Audio", command = lambda:
btn_video = tk.Button(window, text="Play Video", command = lambda:
threading.Thread(target=play_video).start()).
panel = tk.Label(window)
panel.pack()
btn_img.pack(pady=5)
btn_audio.pack(pady=5)
btn_video.pack(pack=5)
window.mainloop()
```

Result:

A multimedia application was successfully created that integrates images, audio, and video functionality into a single interactive interface.

Scanned with OKEN Scanner

Output:

You: hello
You sent an image:
sample_image.jpg
You sent an audio message:
sample_audio.mp3.

15. Capture Video/Audio from webcam or microphone and Display on Multimedia Interface

Aim:

To develop a program that captures Live video & Live audio.

Program:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk
def record_audio():
 duration = 5
 fs=44100
 print("Recording Audio...")
 audio=sd.rec(int(duration*fs),samplerate=fs,channels=2)
 sd.wait()
 write('recorded_audio.wav',fs,audio)
 print("Audio recording saved.")
def capture_video():
  cap=cv2.VideoCapture(0)
  print("Press 'q' to stop video")
  while True:
    ret, frame=cap.read()
    if not ret:
      break
    cv2.imshow('Webcam Feed', frame)
    if cv2.waitKey(1) & 0xFF==ord('q'):
      break
  cap.release()
  cv2.destroyAllWindows()
```

Scanned with OKEN Scanner

```
window = tk.Tk()
window.title("Capture Audio & Video")
btn_audio = tk.Button(window, text="Record Audio", command=lambda:
threading.Thread(target=record_audio).start())
btn_video = tk.Button(window, text="Start Webcam", command=lambda:
threading.Thread(target=capture_video).start())
btn_audio.pack(pady=10)
btn_video.pack(pady=10)
window.mainloop()
```

Result:

Live video and audio were successfully captured using webcam and microphone and displayed.

Scanned with OKEN Scanner